

RAPPORT DE PROJET

Project Meltdown

Serious game multi-agent de gestion de crise

Hackathon SCIA 2026 • Arthur Lefebvre • vérifié le 5 septembre 2026

Résumé

Project Meltdown est un serious game court qui place le joueur dans le rôle d'un Product Owner confronté à une crise projet. Le joueur donne des consignes en langage naturel à quatre personnages pilotés par un modèle, tandis qu'un moteur de règles conserve la maîtrise des conséquences. Le rapport suit une partie complète depuis le briefing jusqu'au débrief : la livraison aboutit en cinq tours avec 100 % de progression, un budget de 34/100, une confiance client de 73/100, un moral de 60/100 et une sécurité vérifiée.

Idée centrale. Pour ce projet, j'ai choisi une architecture hybride : l'IA interprète et propose des intentions, mais elle ne modifie jamais directement les métriques du jeu. Cette séparation apporte une interaction naturelle tout en conservant la traçabilité attendue dans un serious game.

Serious game multi-agent • Décisions en langage naturel • Débrief explicable

1 Concept et expérience de jeu

Le scénario place le Product Owner à trois jours d'une échéance critique. Un défaut technique doit être évalué, une fonctionnalité supplémentaire a été promise et la capacité de l'équipe est limitée. L'objectif est de faire vivre des arbitrages concrets : rechercher l'information, négocier un engagement réaliste et vérifier la sécurité avant de livrer.

Le joueur dispose de six tours et de quatre interlocuteurs complémentaires :

- Alex, responsable du développement, transforme les priorités en travail et signale la surcharge.
- Camille, côté client, révèle le besoin réel et accepte ou refuse les propositions de périmètre.
- Sam, côté commercial, porte la promesse initiale et la communication de l'engagement.
- Morgan, côté sécurité, audite le risque et valide la correction avant une livraison.

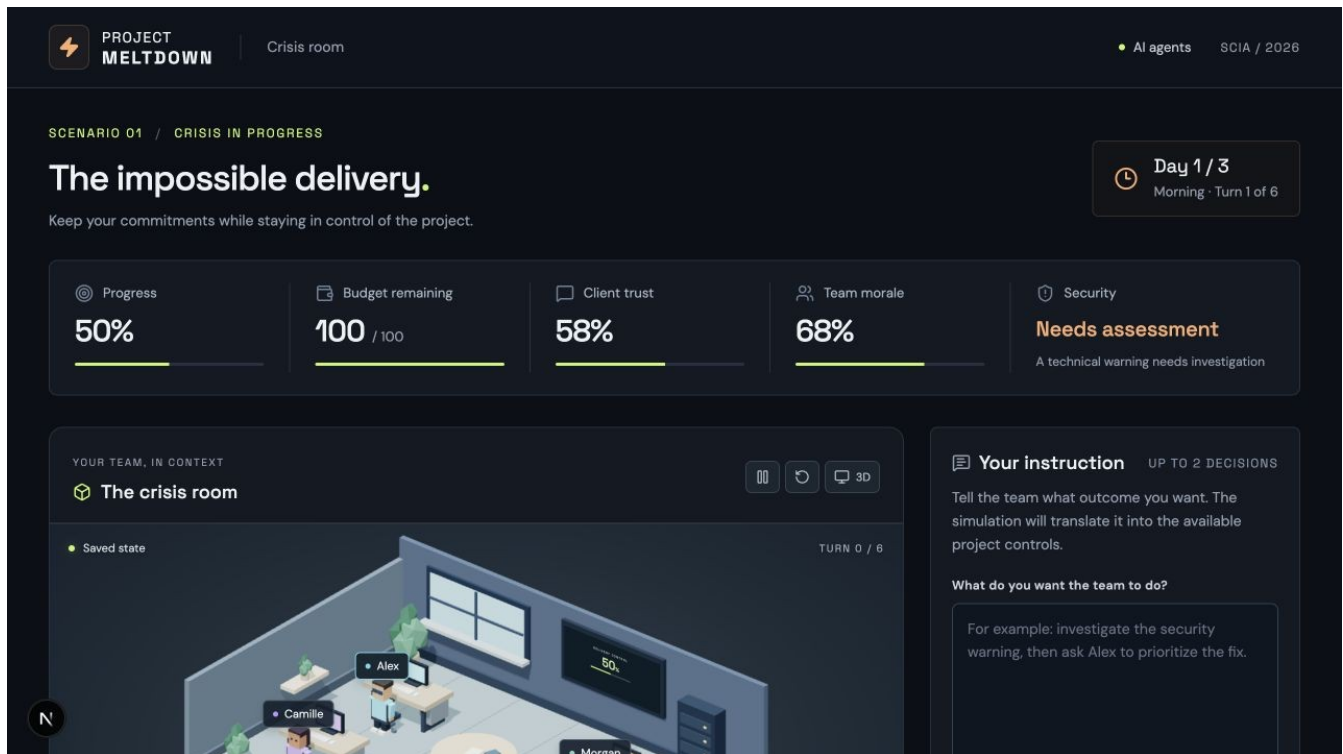


Figure 1 - Briefing, indicateurs, personnages et consigne libre. Source : prototype (jeu-briefing.png).

2 Déroulement d'un parcours

Un parcours de démonstration commence par l'audit de l'alerte et la clarification du besoin client. Le joueur peut ensuite négocier un périmètre réduit, donner la priorité au correctif, faire vérifier la sécurité, terminer le socle puis livrer la version convenue.

Tour	Décision du joueur	Conséquence observée
1	Auditer le défaut et clarifier le besoin	Risque critique confirmé ; une démonstration réduite devient acceptable
2	Prioriser le correctif et négocier	Périmètre réduit accepté ; le correctif progresse
3	Terminer le correctif et communiquer	Correction terminée ; état partagé aux parties prenantes
4	Vérifier la correction et reprendre le socle	Sécurité validée ; socle terminé
5	Livrer le périmètre convenu	Livraison contrôlée et débrief final

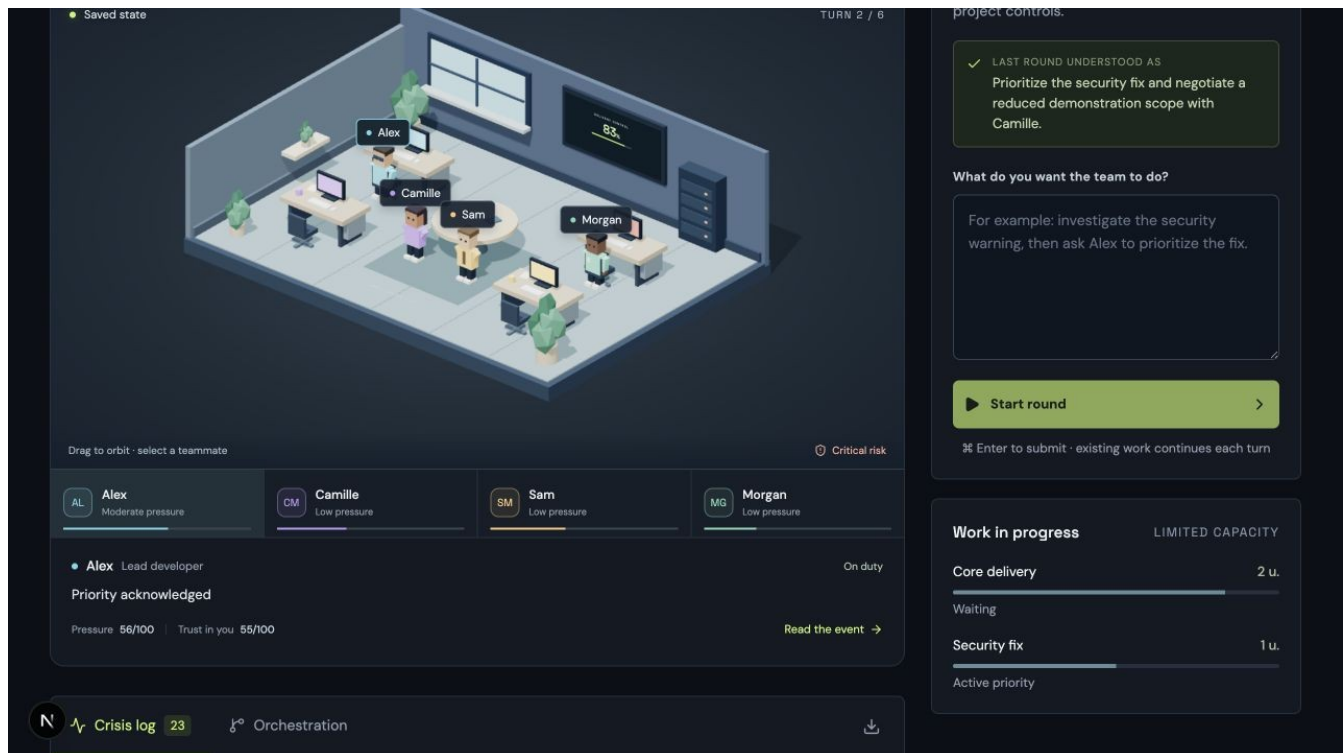


Figure 2 - Tour 2 : consigne interprétée, agents et conséquences. Source : prototype (jeu-tour-2.png).

Le parcours n'est pas une séquence figée dans l'interface : chaque consigne est interprétée, validée, distribuée aux personnages puis résolue dans l'état persistant. Les tâches acceptées continuent également d'un tour à l'autre.

3 Architecture et technologies

Architecture fonctionnelle de Project Meltdown

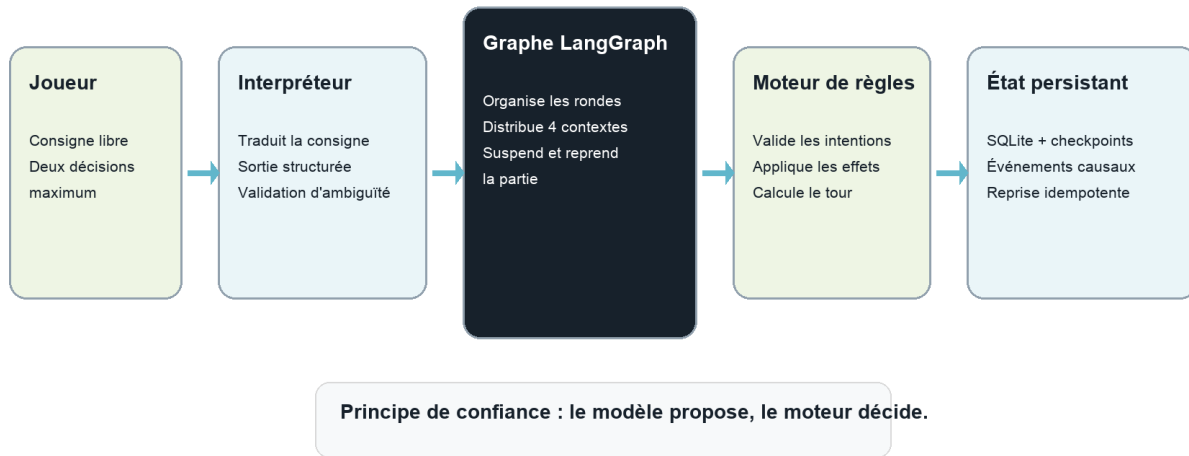


Figure 3 - Chaîne principale de traitement d'une consigne joueur.

L'architecture sépare les responsabilités pour rendre la simulation contrôlable. L'interpréteur convertit une phrase en zéro, une ou deux actions autorisées. LangGraph distribue ensuite un contexte filtré aux personnages. Le moteur vérifie leurs intentions, applique les coûts et fait avancer le temps une seule fois. Enfin, SQLite enregistre la partie et ses checkpoints avant que le navigateur reçoive une projection publique.

Couche	Technologies	Rôle dans le projet
Interface	Next.js 16.3.4, React 19, TypeScript 5.9	Tableau de commandement, saisie libre, polling et débrief
Visualisation	Three.js 0.185.1, React Three Fiber 9.7.0, Drei 10.7.8	Bureau 3D low-poly, personnages et états visuels
API	Python 3.12.13, FastAPI 0.141.1, Pydantic 2.13.5	Contrats HTTP, validation et sérialisation
Orchestration	LangGraph 1.2.11	Interruptions, reprise, distribution Send et deux rondes maximum
Agents	LangChain 1.3.18, OpenAI GPT-5.6 Luna	Intentions structurées, répliques, raisons et émotions des quatre personnages
Persistence	SQLite et checkpointer LangGraph	État canonique, événements, reprise et idempotence
Fiabilité	Pydantic, règles métier et reprise explicite	Validation des sorties, effets contrôlés et reprise après échec

4 Graphe d'orchestration LangGraph

Le graphe ci-dessous représente le cycle réellement implémenté. Il montre les deux interruptions possibles, la distribution parallèle des personnages, la seconde ronde conditionnelle, la finalisation unique du tour et la persistance de l'état.

Graphe LangGraph complet de Project Meltdown

Interruptions joueur, distribution parallèle, boucles de ronde et persistance

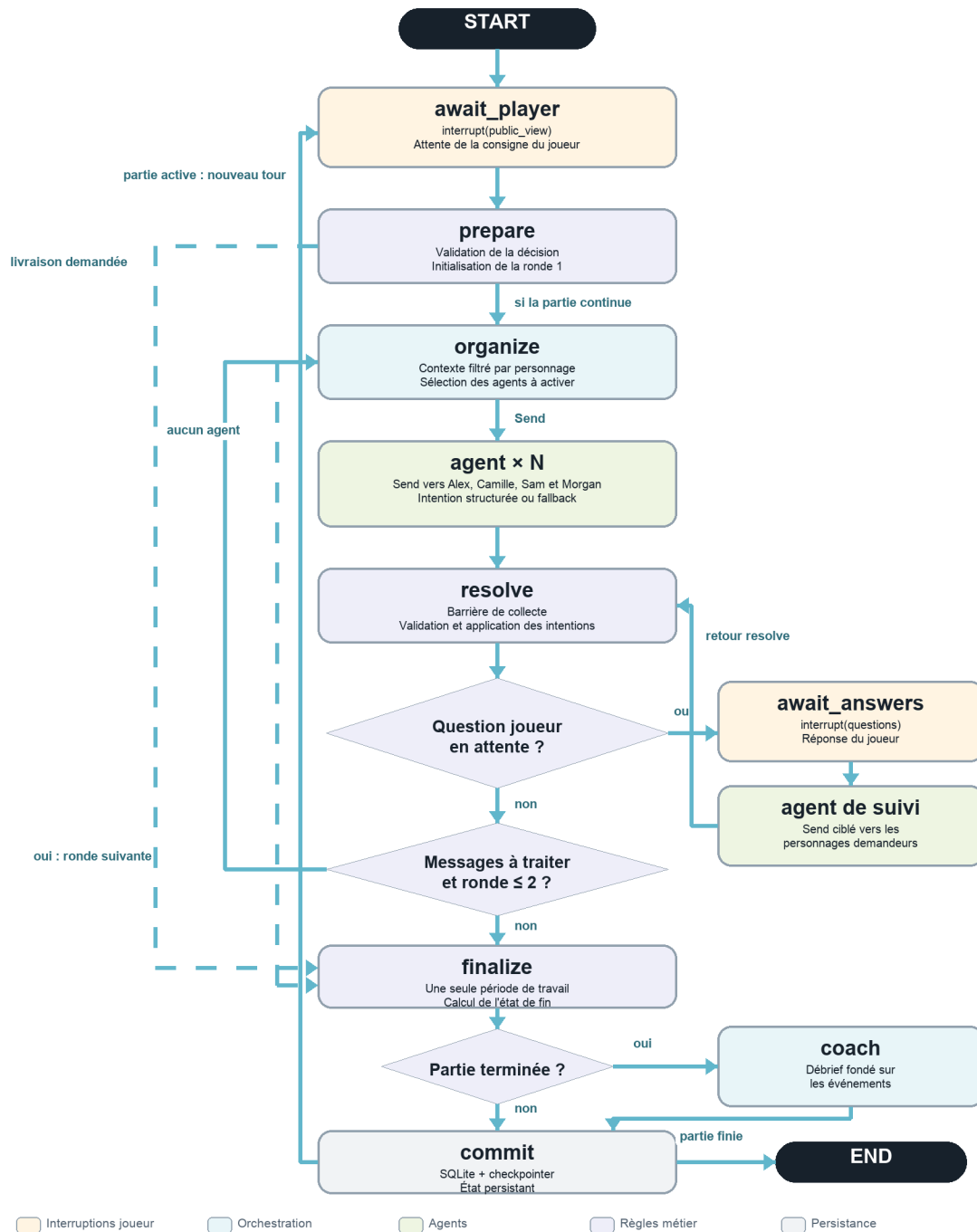


Figure 4 - Graphe d'exécution complet d'un tour de Project Meltdown.

5 Méthodologie de conception

Construction par versions jouables

J'ai construit le projet seul par incréments fonctionnels : cadrage du scénario, moteur métier, orchestration multi-agent, interaction naturelle, puis mise en scène 3D. Chaque étape produisait une version jouable et observable avant l'ajout de la suivante. La validation finale a consisté à parcourir le produit complet dans un navigateur, du briefing au débrief.

1. Cadrer l'objectif pédagogique et écrire l'arbre de décision ainsi que le graphe d'orchestration.
2. Construire le moteur déterministe, l'API, la persistance et le premier tableau de bord.
3. Ajouter l'interprétation des consignes naturelles, les pauses pour questions et la reprise d'une ronde.
4. Calibrer le modèle sur huit situations, corriger les contextes ambigus puis retirer le secours silencieux afin que toute erreur persistante reste visible et récupérable.
5. Intégrer le bureau 3D, l'accessibilité clavier, le mouvement réduit et le fallback 2D.
6. Relier la chronologie, le débrief et les écrans 3D aux mêmes événements persistés.

Approche d'apprentissage

Pour construire la dimension pédagogique, je me suis appuyé sur l'apprentissage par l'action : le joueur décide, observe les conséquences, comprend ses arbitrages puis peut essayer une autre stratégie. Cette logique est traduite dans le produit de la manière suivante :

Étape pédagogique	Traduction dans le jeu
Expérience concrète	Prendre une décision dans une crise courte avec des ressources limitées.
Observation réflexive	Observer les réactions, les métriques et la chronologie causale du tour.
Conceptualisation	Relier les arbitrages entre périmètre, sécurité et équipe.
Expérimentation active	Rejouer la crise en essayant une autre stratégie.

Je me suis également appuyé sur une boucle courte entre décision, action et retour, ainsi que sur un débrief relié aux événements réels de la partie. Chaque conséquence reste ainsi compréhensible et utile pour progresser.

Contrôle de l'IA

- Le modèle ne reçoit que les faits connus par son personnage et une liste d'actions autorisées.
- La sortie est structurée et revalidée par Pydantic et par les règles métier.
- Une sortie invalide ou une erreur persistante interrompt la ronde dans un état sauvegardé ; le joueur peut reprendre explicitement la même ronde avec « Retry safely ».
- Les conséquences et les métriques sont calculées par le moteur, jamais directement par le LLM.
- Le coach final sélectionne des événements déjà rédigés ; chaque référence est vérifiée.

Le runtime requiert un modèle configuré. Le choix local est openai:gpt-5.6-luna avec un effort de raisonnement none, 384 tokens de sortie maximum, un timeout de 20 secondes et une relance fournisseur. Il n'existe plus de mode de personnages à règles ni de secours silencieux.

6 Résultat démontré

Le produit couvre une boucle complète : briefing, consigne naturelle, interprétation visible, réponses des agents, progression persistée, livraison et débrief causal. Les cinq captures de ce rapport montrent ces étapes dans le prototype en fonctionnement.

Validation dans le navigateur

Le 5 septembre 2026, une partie complète a été jouée dans l'interface avec le modèle configuré. La livraison a abouti au cinquième tour avec 100 % de progression, un budget de 34/100, une confiance client de 73/100, un moral de 60/100 et une sécurité vérifiée.

Une consigne volontairement vague a d'abord été refusée sans consommer le tour. Au quatrième tour, une sortie non conforme au schéma Pydantic a interrompu la ronde. L'état est resté sauvegardé et le bouton « Retry safely » a repris la même ronde sans doubler le temps ni les coûts. La seconde tentative a réussi, ce qui démontre la reprise prévue par le produit.

Deux vues dédiées rendent désormais les mécanismes internes directement exploitables dans un rapport : l'une rassemble les réponses contextualisées des personnages ; l'autre expose la chaîne d'orchestration et sa trace d'exécution.

LATEST AGENT ROUND 4 GROUNDED RESPONSES

Agent perspectives on your decision

Each character reacts from a distinct role, emotional state, and private context.

Your direction → AI agent perspectives → Rule-based consequences

Alex · Development
TURN 1 · ROUND 2 URGENT

"I'm prioritizing the export vulnerability fix. Two fix work units remain; at the normal fix throughput of one unit per period, that is roughly two periods before security verification can begin. The critical vulnerability must be fixed and verified before release."

WHY The audit confirmed a critical export-module vulnerability, and the assigned fix still has 2 work units remaining.

Camille · Client
TURN 1 · ROUND 1 CONCERNED

"For the demonstration, I need the promised extra feature shown in a working end-to-end flow, with the core existing functionality still stable. Please confirm what can realistically be demonstrated before we commit to timing."

WHY I am clarifying the actual demonstration need before technical capacity and timing are confirmed, as directed. The sales lead promised an extra feature prematurely.

trust +4

Sam · Sales
TURN 1 · ROUND 2 CONCERNED

"I need to reset expectations: I promised an extra feature before confirming technical capacity. Since the deadline is for a demonstration, I propose we explicitly agree on a reduced scope for the demo and treat the extra feature as a later commitment. Please confirm whether that works for you."

WHY My earlier promise was made without technical confirmation, while the client has stated that a reduced-scope demonstration is acceptable if explicitly agreed.

Morgan · Security
TURN 1 · ROUND 1 CONCERNED

"I'm auditing the export-module defect now to establish its severity and provide a basis for the fix."

WHY The defect requires a security audit, and the directive explicitly assigns me to audit it. This is the useful next intervention.

Figure 5 - Réponses contextualisées des quatre agents : rôle, émotion, parole, justification et effets visibles. Source : prototype (jeu-agents.png).

Une partie de capture complémentaire a produit deux rondes et sept activations au premier tour. Cette trace illustre la distribution réelle des personnages sans exposer leurs connaissances privées.

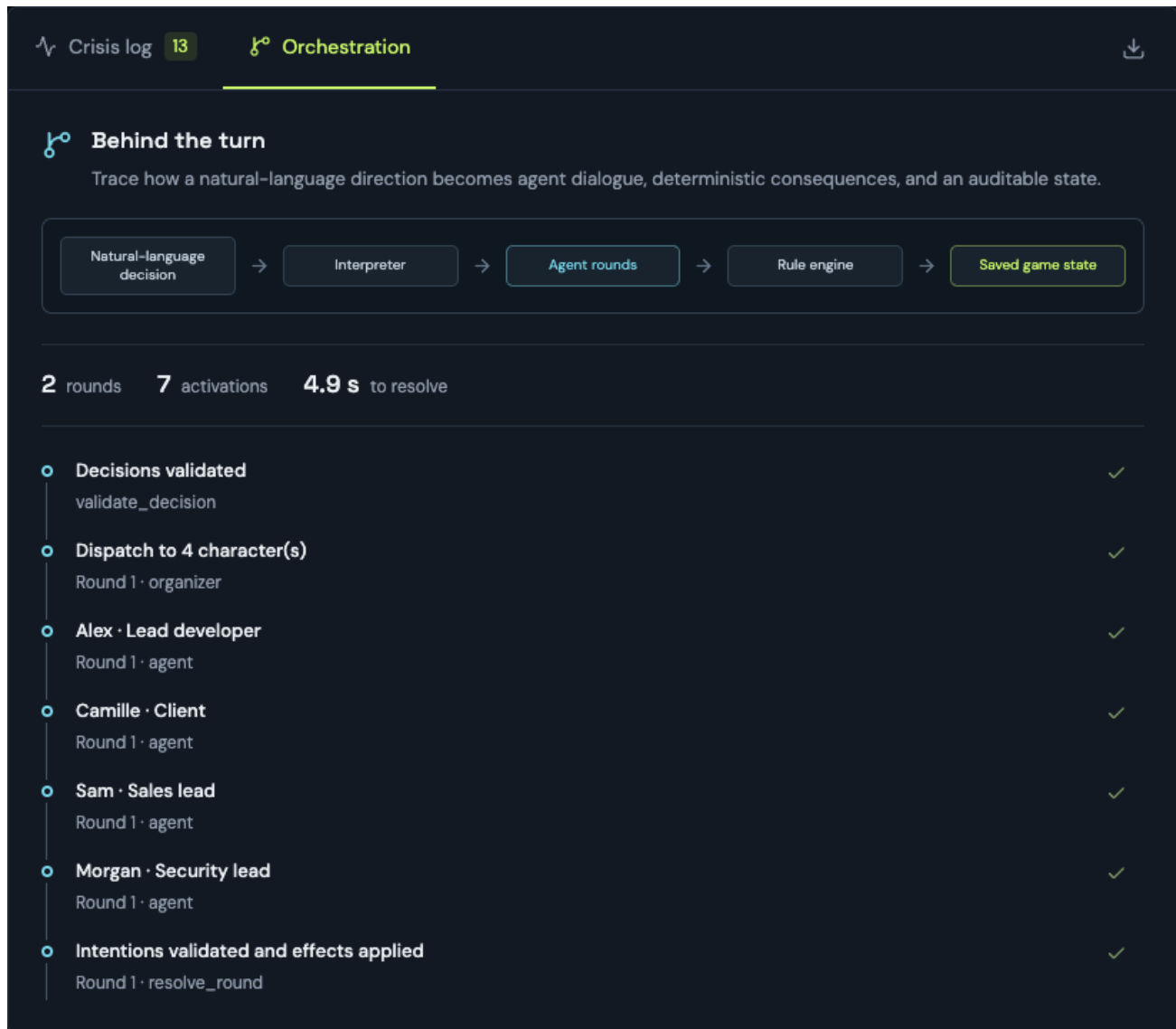


Figure 6 - Pipeline d'un tour : décision libre, interprétation, rondes d'agents, moteur de règles et sauvegarde. Trace observée : 2 rondes, 7 activations, 4,9 s (jeu-orchestration.png).

Indicateur	Résultat du parcours	Lecture
Progression	100 %	Périmètre convenu terminé
Budget	34 / 100	Coûts des décisions et des cinq périodes
Confiance client	73 / 100	Besoin clarifié, périmètre accepté et état communiqué
Moral	60 / 100	Équipe restée opérationnelle sous pression
Sécurité	Vérifiée	Correctif terminé puis contrôlé avant livraison

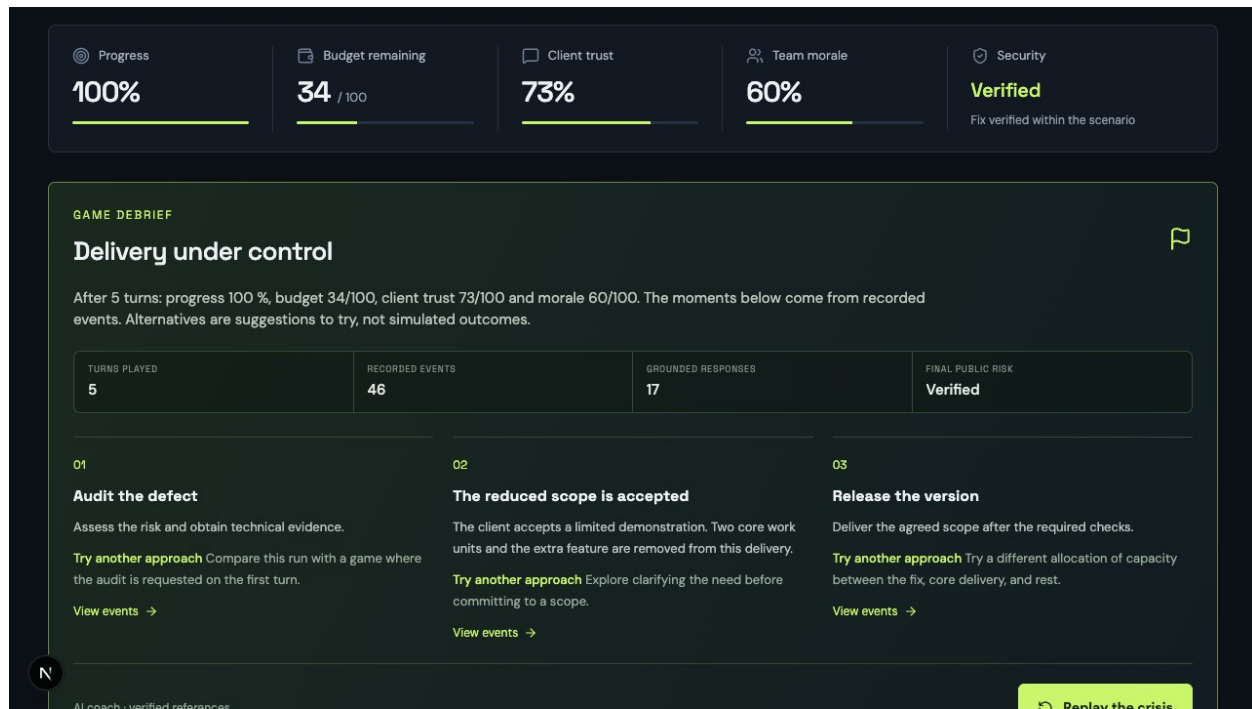


Figure 7 - Débrief final : métriques, 46 événements, 17 réponses contextualisées et trois moments reliés à leurs preuves. Source : prototype (jeu-bilan.png).

7 Conclusion et sources

La partie observée aboutit à une livraison contrôlée en cinq tours, avec le périmètre terminé et la sécurité vérifiée. Ce résultat montre l'apport principal de Project Meltdown : le joueur conserve la liberté du langage naturel, tandis que les rôles séparés, les règles métier, la persistance et les événements sources rendent chaque conséquence explicable. Le prototype réunit ainsi une expérience jouable, une architecture multi-agent et un objectif pédagogique dans un même parcours documenté.

Sources techniques du rapport

- Code source : backend/meltdown/graph.py, engine.py, agents.py, interpreter.py, service.py, projection.py et backend/tests/.
- Interface : web/components/dashboard.tsx, web/components/office/, web/lib/ et web/tests/.
- Conception et méthode : docs/graphe-orchestration.md, docs/arbre-decisionnel.md, docs/model-calibration.md et docs/journal.md.
- Captures du prototype : cinq écrans dans docs/report-assets/ — briefing, tour 2, agents, orchestration et bilan ; dépendances vérifiées dans uv.lock et web/package-lock.json.

Documentation officielle : [LangGraph](#) · [FastAPI](#) · [Next.js](#) · [React Three Fiber](#)

Modèle configuré : [documentation officielle OpenAI de GPT-5.6 Luna](#) (sorties structurées prises en charge).

Sources sur l'apprentissage

Kolb, D. A. (1984). *Experiential Learning: Experience as the Source of Learning and Development*. Prentice Hall. Le cycle est utilisé ici comme cadre de conception de l'expérience.

Garris, R., Ahlers, R. et Driskell, J. E. (2002). [Games, Motivation, and Learning: A Research and Practice Model](#). *Simulation & Gaming*, 33(4), 441-467.

Crookall, D. (2010). [Serious Games, Debriefing, and Simulation/Gaming as a Discipline](#). *Simulation & Gaming*, 41(6), 898-920.